

SILICON DREAMS

Study Guide

MODULE 3 · WEEK 3 · CM-HW-101-M3

SG-M3-07 pairs with TB-M3-07

LVS and Timing Closure

Diagnose the three canonical LVS fail patterns; close timing on the worst corner with two of four techniques.

Learning objectives

- Read an LVS report and classify the fail into one of three patterns: port name mismatch, net connectivity mismatch, cell count mismatch.
- Understand setup, hold, and why both corners matter.
- Apply two of four timing closure techniques: tighten SDC, re-structure RTL critical path, upsize flop drive strength, or bump CLOCK_PERIOD.
- Achieve worst-corner WNS ≥ 0 and hold slack ≥ 0 .

Step 1 · Read the LVS report

```
cd runs/<latest-tag>/reports/signoff
less lvs.rpt

# Top of a healthy report looks like:
#   Circuit 1: layout
#   Circuit 2: schematic
#   Circuits match: MATCH
#
# An unhealthy one:
#   Port mismatch on "axiom_misbehaviour":
#     layout has "axiom_misbehaviour" as uio_out[1]
#     schematic has "axiom_misbeh_out" as uio_out[1]
```

Step 2 · Pattern A — Port name mismatch

Symptom: LVS fails with 'Port mismatch on <name>'. Cause: you renamed a Verilog port but the pad ring / pin_order config still uses the old name.

```
# Fix: update librelane/pin_order.cfg
# BEFORE:
#   uio_out[1] axiom_misbeh_out
# AFTER:
#   uio_out[1] axiom_misbehaviour

# Re-run
./scripts/run-librelane.sh
#check thr lvs-report
runs/reports/signoff/sta-worst.rpt
```

Step 3 · Pattern B — Net connectivity mismatch

Symptom: 'Net <N> has different topology in layout vs schematic'. Cause: a `wire` was left dangling because a port on a submodule is not hooked up in top.v.

```
# Find it quickly:
grep -n "assign" src/top.v | grep uio_out
# Look for missing assigns or conflicting drivers.

# Classic case: you declared uio_out[3] as a debug output but never drove it.
# Fix in top.v:
assign uio_out[3] = arbiter_grant_axiom;
assign uio_oe[3] = 1'b1;
```

Step 4 · Pattern C — Cell count mismatch

Symptom: 'Circuit 1 has 1247 cells; Circuit 2 has 1245 cells'. Cause: synthesis inserted a cell (typically a buffer for fanout) that the schematic extraction didn't see, OR a constant has been optimised away in one but not the other.

Fix: usually a re-run after fixing Pattern A or B resolves it. If persistent, the issue is in the liberty file or a missing model for a non-standard cell. Check `logs/signoff/magic-extract.log` for unresolved references.

Step 5 · Read the timing report

```
less runs/<tag>/reports/signoff/sta-worst.rpt

# Key fields:
# Startpoint: <flop_X>
# Endpoint:   <flop_Y>
# Path Type:  max (setup) or min (hold)
```

```
# required time: 10.00 ns
# arrival time: 10.32 ns
# slack: -0.32 ns <-- NEGATIVE = fail
```

At the bottom of the report, OpenSTA prints WNS (worst negative slack). $WNS \geq 0$ means you've closed setup.

Step 6 · Timing closure technique 1 — tighten SDC

Before anything else, make sure your SDC is not LYING. Common lie: input delay too low. Real-world inputs arrive with ~2 ns of external setup — model it:

```
set_input_delay -clock clk -max 2.0 [get_ports {ui_in[*] uio_in[*]}]
set_output_delay -clock clk -max 2.0 [get_ports {uo_out[*] uio_out[*]}]
```

Tightening the SDC does NOT make timing worse — it makes it HONEST. If WNS was +0.5 ns under a loose SDC but -1.5 ns under a tight one, you never had 0.5 ns to begin with.

Step 7 · Technique 2 — Restructure the critical path

Open the sta-worst.rpt and look at the path. If it's 15+ cells deep, you probably have a long combinational function — break it:

```
// BEFORE — one cycle, long path
always @(posedge clk)
    result <= (a & b) | (c & d) | (e & f) | (g & h) | (i & j) | (k & l);

// AFTER — two cycles, balanced path
reg stage1_ab, stage1_cd, stage1_ef, stage1_gh, stage1_ij, stage1_kl;
always @(posedge clk) begin
    stage1_ab <= a & b; stage1_cd <= c & d; stage1_ef <= e & f;
    stage1_gh <= g & h; stage1_ij <= i & j; stage1_kl <= k & l;
    result <= stage1_ab | stage1_cd | stage1_ef
              | stage1_gh | stage1_ij | stage1_kl;
end
```

E8873C

Adding a pipeline stage changes latency. Make sure the rest of your design tolerates it — the integration smoke test in test/top/ will catch most violations.

Step 8 · Technique 3 — Upsize flop drive

Some flops have multiple drive strengths. Yosys picks the smallest by default; if the path is dominated by the flop-to-flop wire, upsizing the source flop from `\_1` to `\_4` can pay off.

```
# In config.json
"SYNTH_STRATEGY": "DELAY 4",
"SYNTH_BUFFERING": 1,
```

```
"SYNTH_SIZING": 1
```

Step 9 · Technique 4 — Bump CLOCK_PERIOD (last resort)

If all else fails, raise CLOCK_PERIOD from 10 ns to 12 ns (83 MHz → 100 MHz). You lose the 100 MHz badge but you ship the chip.

```
# config.json
"CLOCK_PERIOD": 12.0
```

Grader rules: 10 ns earns full points; 12 ns is -50 XP; 15 ns is -150 XP; anything slower is a fail.

Step 10 · Close hold

Hold failures are less common but much worse — they cannot be fixed by slowing the clock. Symptom in sta-worst.rpt:

```
Path Type: min (hold)
slack: -0.08 ns <-- hold violation
```

Fix: add buffers on the offending path. LibreLane does this automatically if you set:

```
"RSZ_HOLD_MAX_BUFFER_PERCENT": 50
```

If the post-run report still shows hold fails, your reset synchroniser is probably the culprit — it has a tight hold window by design. Widen it to 3-flop stages.

Troubleshooting

LVS MATCH but timing fails by 0.01 ns	Tighten SDC a bit more then re-run. 10 ps of slack is real but bootstrappable.
LVS fails AFTER timing was clean	You changed a port name during closure. Update pin_order.cfg.
Hold fix breaks setup	RSZ_HOLD_MAX_BUFFER_PERCENT is too aggressive. Step down from 50 → 30.
OpenSTA crashes	Usually a malformed SDC. Run `sta -no_init` and source your sdc manually to see the parse error.

Reflection

- Record WNS-before and WNS-after for each technique you applied. This is a rich escape-log entry.
- Which technique gave you the biggest bang for the buck? Most learners: restructure (Step 7) > SDC tighten (Step 6) > upsize (Step 8) > period bump (Step 9).